

Project report

Implementation of 4 bit synchronous sequential
up/down counter using JKFF

Team “JADA”

Diyorbek Mansurov U2210134 003

Javohirbek Xatamov U2210251 003

Amir Yuldashev U2210257 003

Zikirillayev Asadtilla U2210280 003

Inha university in Tashkent.
DLC SOC 2080.
Section 003.
Team "JADA".
June 2 2024.

Table of contents

1 Introduction.....	4
2 Body.....	5
2.1 State table & K-map.....	7
2.2 Circuit diagram.....	14
2.3 Timing diagram.....	15
2.4 Implementation in TinkerCad.....	17
2.5 Login info.....	17
3 Conclusion.....	17
4 References.....	18

Introduction

The main objective of the project was to design a 4 bit synchronous sequential up or down counter (function of the circuit changes based on the controlling input) using JK flip flops and implement it in Tinkercad.

The model must include a 7 segment display which is to be used for displaying the result of the counter. The cost of the prototype is not important but should not be neglected.

This report describes the designing process of the given task along with the supporting materials and the results of the milestones achieved and in the end the final outcome.

Body

The first thing we did was break down the problem into smaller pieces. As the main goal is to build a counter here is the mind map(or flow chart) of the things necessary in order to accomplish the current task depicted in Figure 1.1:

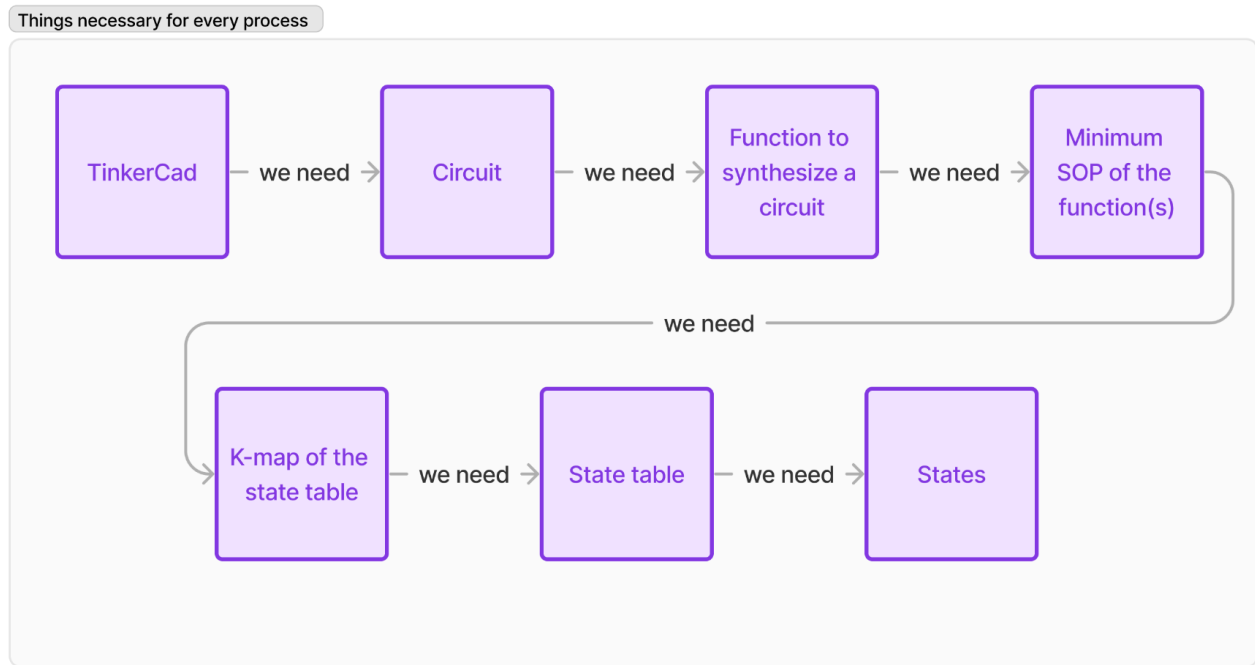


Figure 1.1

First, let's consider the states. Since there are two counting modes (up and down), we created separate state diagrams for each mode. The selection between these modes is based on the controlling input. Consequently, we have two state tables: one for the up counter ($w = 1$), shown in Figure 1.2, and one for the down counter ($w = 0$), shown in Figure 1.3.

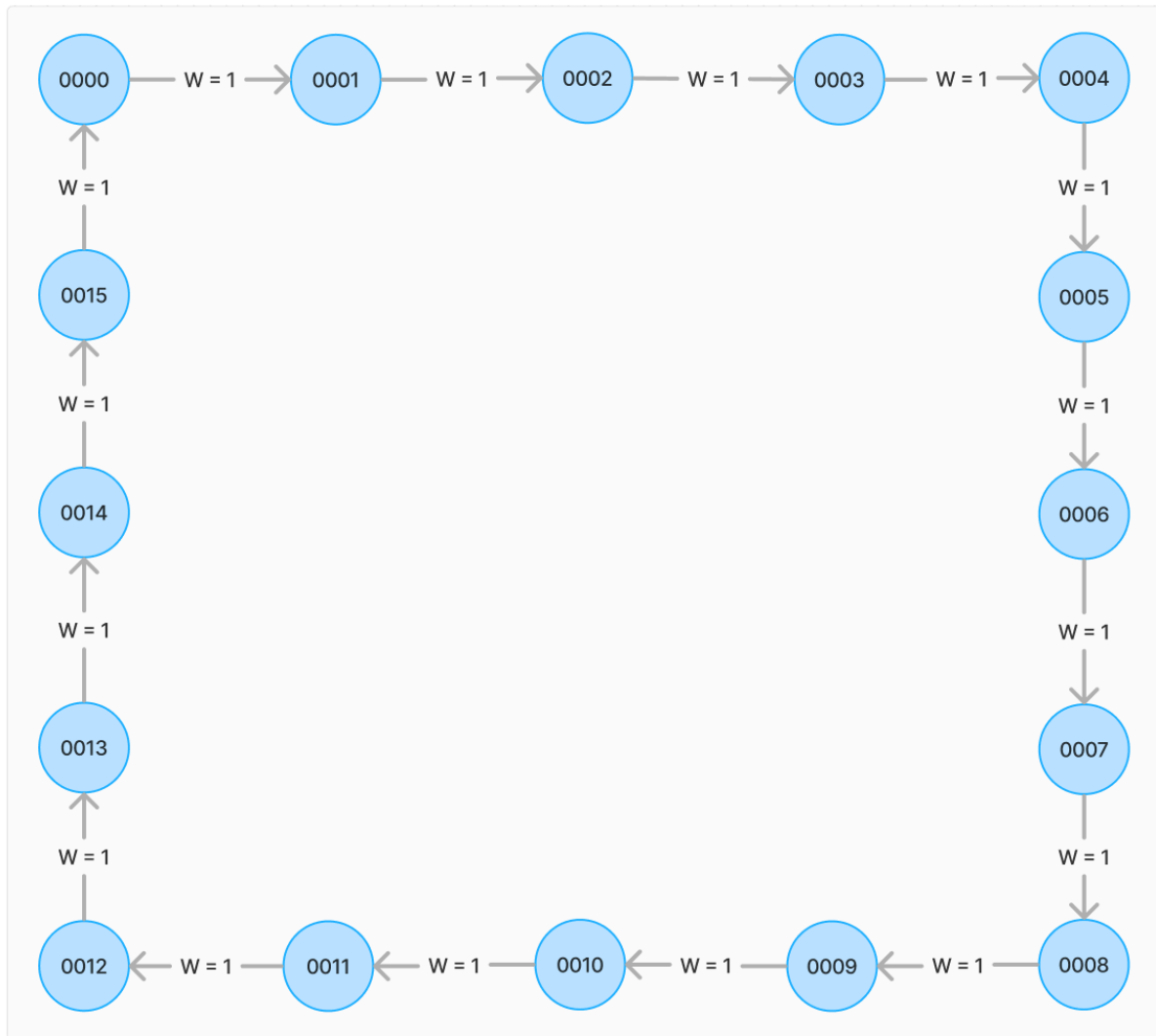


Figure 1.2

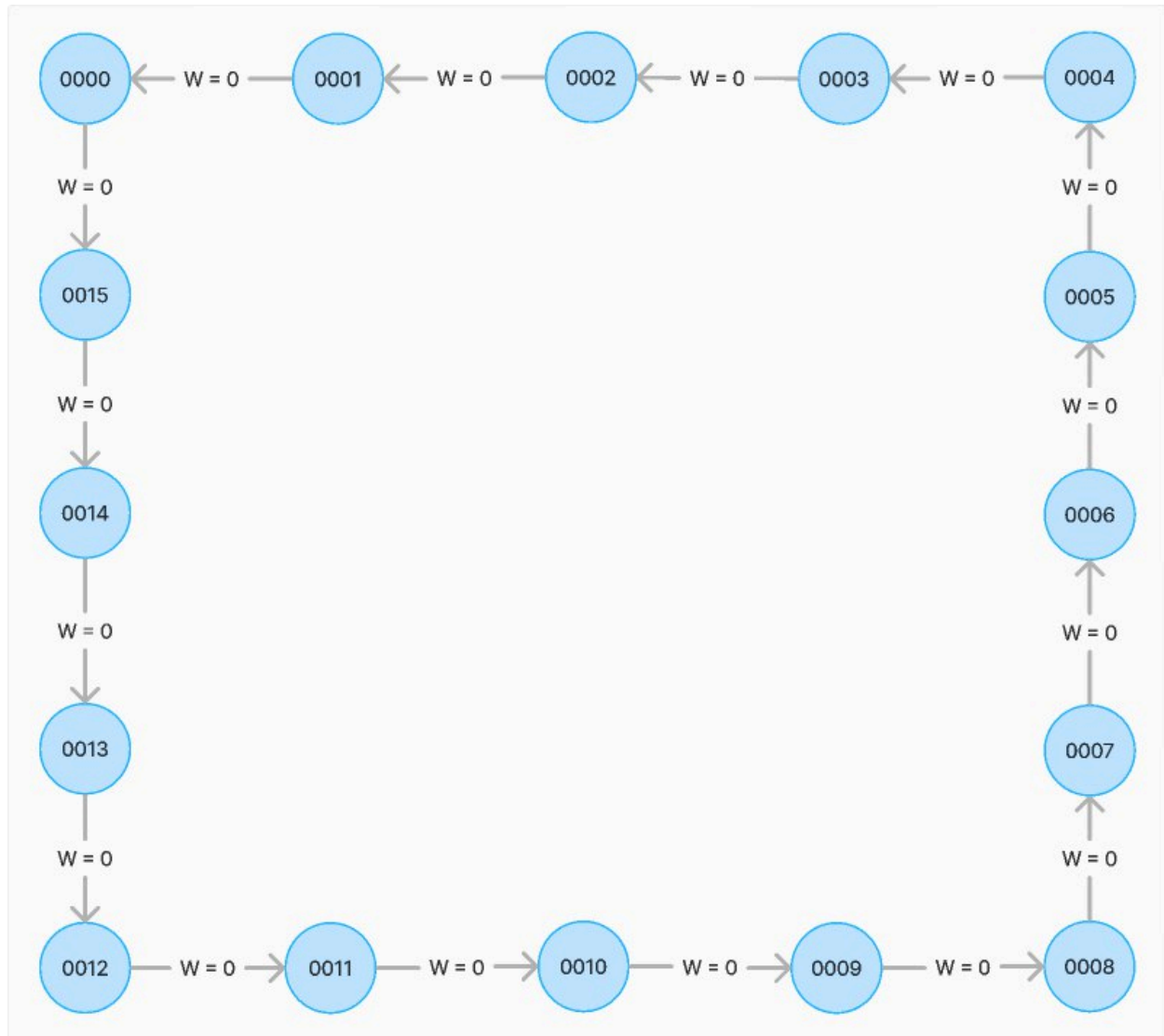


Figure 1.3

2.1 State table & K-map

As the state diagrams are ready, a state table can be built based on them. The one below (Figure 2.1.1) is for $w=0$ which sets the circuit to count down, i.e. all values' next state is their decrement.

Next we did a similar state table for the up counter, the only difference from the previous counter is that for every value of the current state, the next state will be its increment, this can be observed in the Figure 2.1.2.

Current State				Next state (w=0)			
y3	y2	y1	y0	Y3	Y2	Y1	Y0
0	0	0	0	1	1	1	1
0	0	0	1	0	0	0	0
0	0	1	0	0	0	0	1
0	0	1	1	0	0	1	0
0	1	0	0	0	0	1	1
0	1	0	1	0	1	0	0
0	1	1	0	0	1	0	1
0	1	1	1	0	1	1	0
1	0	0	0	0	1	1	1
1	0	0	1	1	0	0	0
1	0	1	0	1	0	0	1
1	0	1	1	1	0	1	0
1	1	0	0	1	0	1	1
1	1	0	1	1	1	0	0
1	1	1	0	1	1	0	1
1	1	1	1	1	1	1	0

Figure 2.1.1

Current State				Next state (w = 1)			
y3	y2	y1	y0	Y3	Y2	Y1	Y0
0	0	0	0	0	0	0	1
0	0	0	1	0	0	1	0
0	0	1	0	0	0	1	1
0	0	1	1	0	1	0	0
0	1	0	0	0	1	0	1
0	1	0	1	0	1	1	0
0	1	1	0	0	1	1	1
0	1	1	1	1	0	0	0
1	0	0	0	1	0	0	1
1	0	0	1	1	0	1	0
1	0	1	0	1	0	1	1
1	0	1	1	1	1	0	0
1	1	0	0	1	1	0	1
1	1	0	1	1	1	1	0
1	1	1	0	1	1	1	1
1	1	1	1	0	0	0	0

Figure 2.1.2

As the values of y and Y are basically Q(t) and Q(t+1), they can be used to determine the values of J and K, below is the table which will be used for this (Figure 2.1.3)

Q(t)	Q(t+1)	J	K
0	0	0	d
0	1	1	d
1	0	d	1
1	1	d	0

Figure 2.1.3

w = 0							
Y3		Y2		Y1		Y0	
J	K	J	K	J	K	J	K
1	d	1	d	1	d	1	d
0	d	0	d	0	d	d	1
0	d	0	d	d	1	1	d
0	d	0	d	d	0	d	1
0	d	d	1	1	d	1	d
0	d	d	0	0	d	d	1
0	d	d	0	d	1	1	d
0	d	d	0	d	0	d	1
d	1	1	d	1	d	1	d
d	0	0	d	0	d	d	1
d	0	0	d	d	1	1	d
d	0	0	d	d	0	d	1
d	0	d	1	1	d	1	d
d	0	d	0	0	d	d	1
d	0	d	0	d	1	1	d
d	0	d	0	d	0	d	1

Figure 2.1.4

w = 1							
Y3		Y2		Y1		Y0	
J	K	J	K	J	K	J	K
0	d	0	d	0	d	1	d
0	d	0	d	1	d	d	1
0	d	0	d	d	0	1	d
0	d	1	d	d	1	d	1
0	d	d	0	0	d	1	d
0	d	d	0	1	d	d	1
0	d	d	0	d	0	1	d
1	d	d	1	d	1	d	1
d	0	0	d	0	d	1	d
d	0	0	d	1	d	d	1
d	0	0	d	d	0	1	d
d	0	1	d	d	1	d	1
d	0	d	0	0	d	1	d
d	0	d	0	1	d	d	1
d	0	d	0	d	0	1	d
d	1	d	1	d	1	d	1

Figure 2.1.4

The above tables in Figure 2.1.4 and Figure 2.1.5 are the values J and K which were extracted from the state table. The next step would be to make karnaugh maps of J and K as the values are now known in order to determine minimum sop for every J and K. There are two tables for JK values of every y, thus there should be two k-maps for every y.

w = 0									
Y0 J0	1				Y0 K0	1			
y1y0\y3y2	00	01	11	10	y1y0\y3y2	00	01	11	10
00	1	1	1	1	00	d	d	d	d
01	d	d	d	d	01	1	1	1	1
11	d	d	d	d	11	1	1	1	1
10	1	1	1	1	10	d	d	d	d
Y1 J1	$\sim y0$				Y1 K1	$\sim y0$			
y1y0\y3y2	00	01	11	10	y1y0\y3y2	00	01	11	10
00	1	1	1	1	00	d	d	d	d
01	0	0	0	0	01	d	d	d	d
11	d	d	d	d	11	0	0	0	0
10	d	d	d	d	10	1	1	1	1
Y2 J2	$\sim y1^* \sim y0$				Y2 K2	$\sim y1^* \sim y0$			
y1y0\y3y2	00	01	11	10	y1y0\y3y2	00	01	11	10
00	1	d	d	1	00	d	1	1	d
01	0	d	d	0	01	d	0	0	d
11	0	d	d	0	11	d	0	0	d
10	0	d	d	0	10	d	0	0	d
Y3 J3	$\sim y1^* \sim y2^* \sim y0$				Y3 K3	$\sim y2^* \sim y1^* \sim y0$			
y1y0\y3y2	00	01	11	10	y1y0\y3y2	00	01	11	10
00	1	0	d	d	00	d	d	0	1
01	0	0	d	d	01	d	d	0	0
11	0	0	d	d	11	d	d	0	0
10	0	0	d	d	10	d	d	0	0

Figure 2.1.6

w = 1									
Y0 J0	1				Y0 K0	1			
y1y0\y3y2	00	01	11	10	y1y0\y3y2	00	01	11	10
00	1	1	1	1	00	d	d	d	d
01	d	d	d	d	01	1	1	1	1
11	d	d	d	d	11	1	1	1	1
10	1	1	1	1	10	d	d	d	d
Y1 J1	y0				Y1 K1	y0			
y1y0\y3y2	00	01	11	10	y1y0\y3y2	00	01	11	10
00	0	0	0	0	00	d	d	d	d
01	1	1	1	1	01	d	d	d	d
11	d	d	d	d	11	1	1	1	1
10	d	d	d	d	10	0	0	0	0
Y2 J2	y1*y0				Y2 K2	y1*y0			
y1y0\y3y2	00	01	11	10	y1y0\y3y2	00	01	11	10
00	0	d	d	0	00	d	0	0	d
01	0	d	d	0	01	d	0	0	d
11	1	d	d	1	11	d	1	1	d
10	0	d	d	0	10	d	0	0	d
Y3 J3	y2*y1*y0				Y3 K3	y2*y1*y0			
y1y0\y3y2	00	01	11	10	y1y0\y3y2	00	01	11	10
00	0	0	d	d	00	d	d	0	0
01	0	0	d	d	01	d	d	0	0
11	0	1	d	d	11	d	d	1	0
10	0	0	d	d	10	d	d	0	0

Figure 2.1.7

Notes on the above tables Figure 2.1.6 and Figure 2.1.7:

1) Areas marked with red are the values we selected for function extraction.

2) In functions, “*” means “AND”, “+” means “OR” and “~” means “NOT”.

3) y_1y_0 are for columns and y_3y_2 are for rows

2.2 Circuit diagram

On the previous step there are tables with K-maps and functions derived from this map, these functions are shown below for clarification:

$$J_0 = 1$$

$$K_0 = 1$$

$$J_1 = \neg w * \neg y_0 + y_0 * w = \neg(\text{xor}(y_0, w))$$

$$K_1 = \neg w * \neg y_0 + y_0 * w = \neg(\text{xor}(y_0, w))$$

$$J_2 = \neg w * \neg y_0 * \neg y_1 + w * y_0 * y_1$$

$$K_2 = \neg w * \neg y_0 * \neg y_1 + w * y_0 * y_1$$

$$J_3 = \neg w * \neg y_0 * \neg y_1 * \neg y_2 + w * y_0 * y_1 * y_2$$

$$K_3 = \neg w * \neg y_0 * \neg y_1 * \neg y_2 + w * y_0 * y_1 * y_2$$

Figure 2.2.1

Functions are known, so the next step would be to build a circuit, here it is, built in Logisim app (Figure 2.2.2)

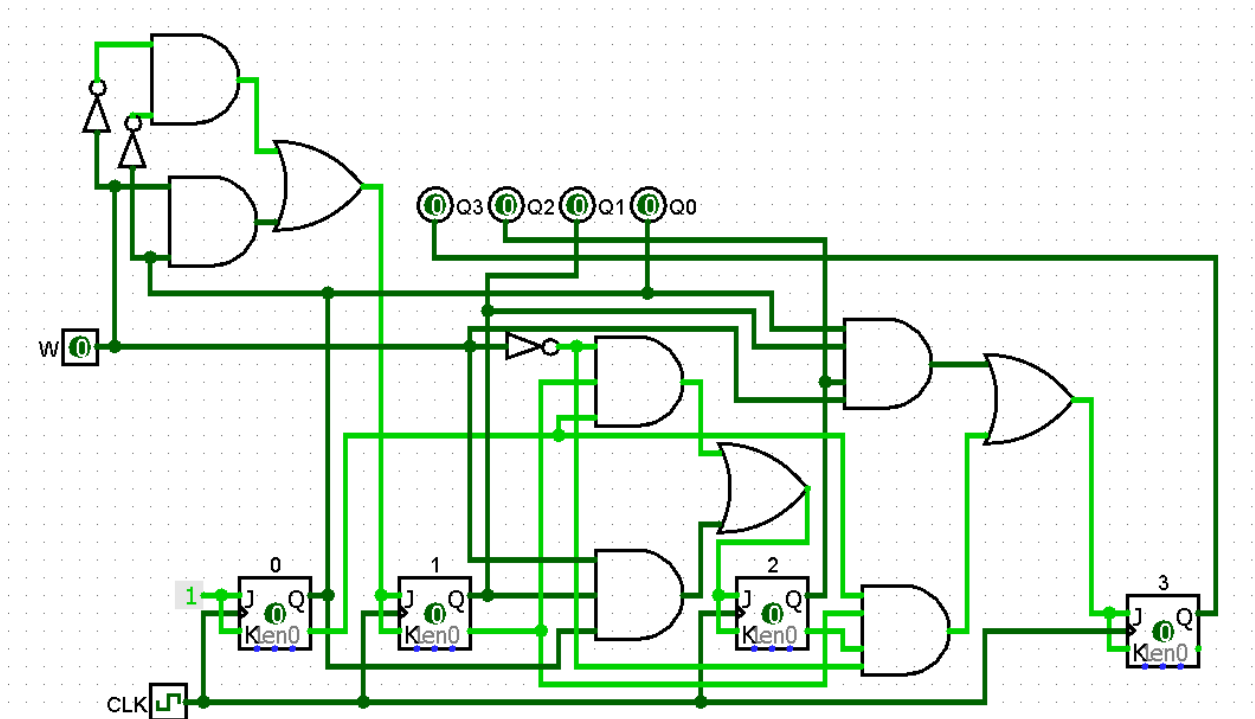


Figure 2.2.2

2.3 Timing diagram

When the circuit is done it's timing diagram can be drawn

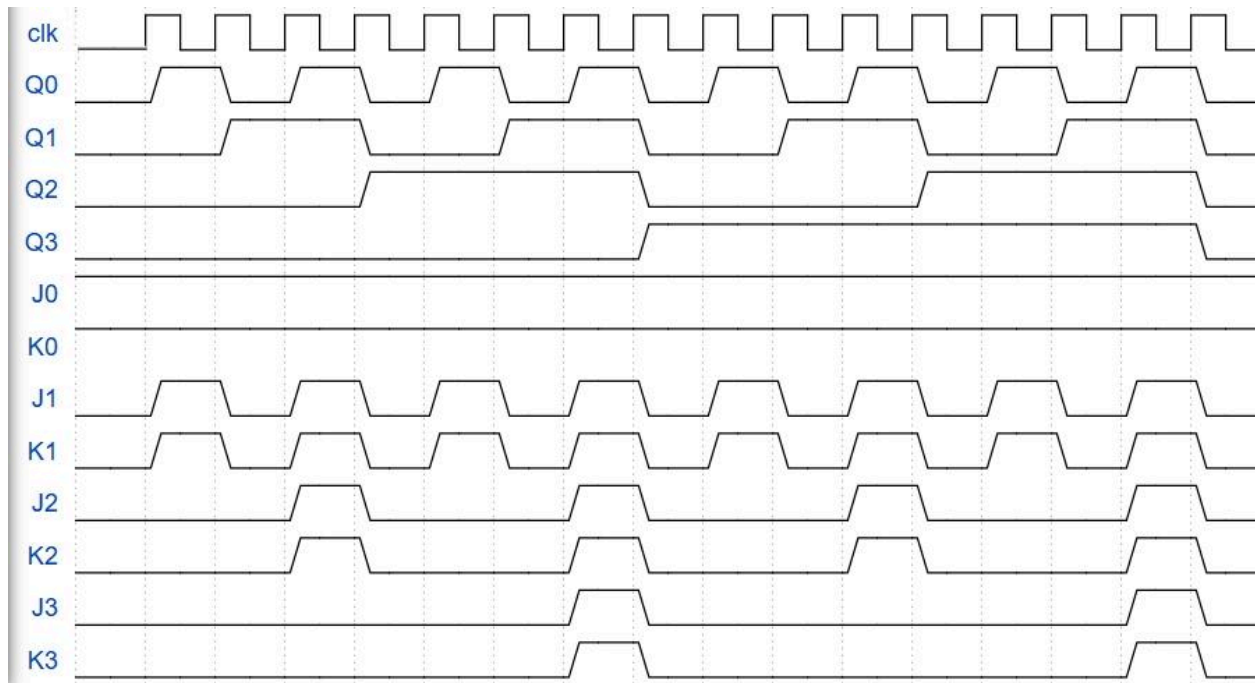


Figure 2.3.1

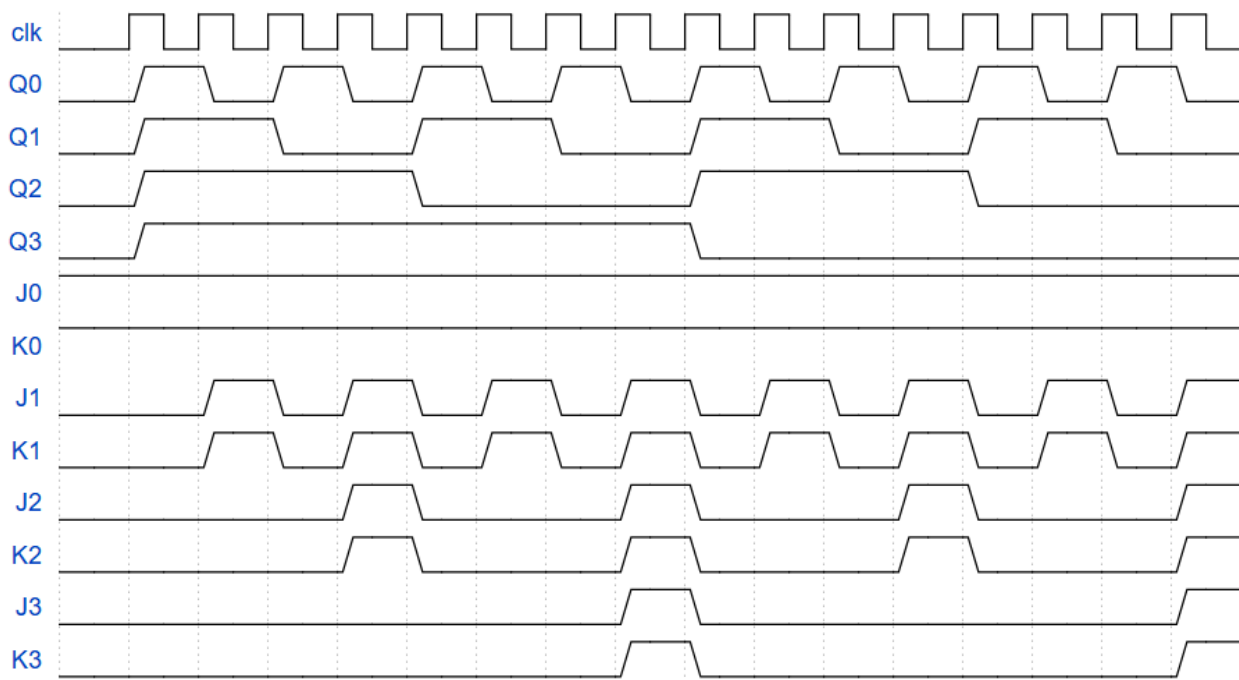


Figure 2.3.2

These are the timing diagrams for the circuit when $w = 1$ (Figure 2.3.1) and $w = 0$ (Figure 2.3.2). It's also worth taking into account that our JKFF is positive-edge triggered.

2.4 Implementation in TinkerCad

Here is our model in TinkerCad, implemented with two 7 segment displays using BCD. It counts from 0 to 15 in the decimal system. The counting of the system can also be observed from the lamps

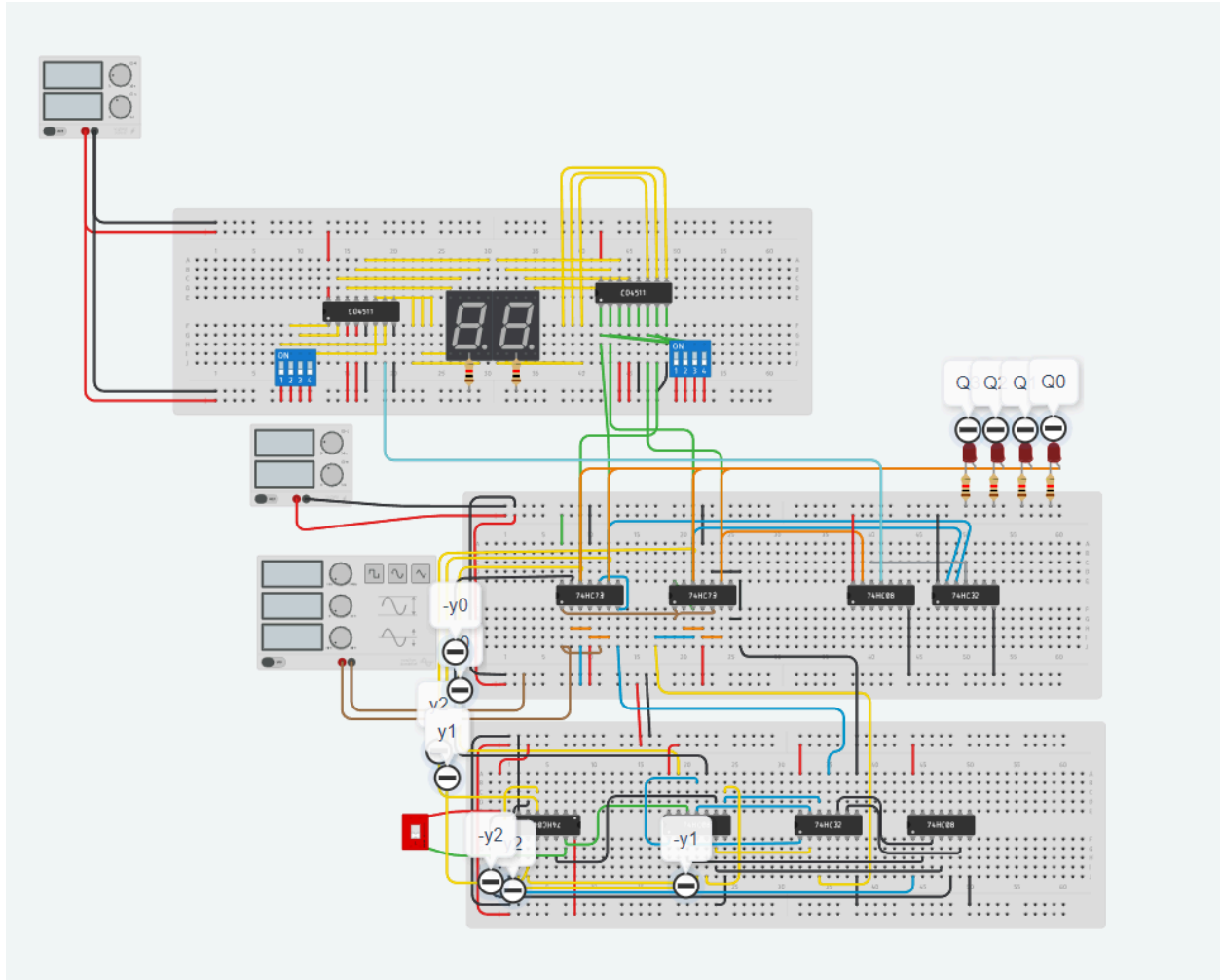


Figure 2.4.1

2.5 Login info

Login: amir1008047@gmail.com

Password: qb12345678

Conclusion

In this project we implemented a 4 bit synchronous sequential up/down counter using 4 JK flip-flops in order to count up to 16 and display it on a 7 segment display. Initially, we defined the states followed by filling a state table and JK values. Then, based on the data obtained, we derived the minimum SOP functions using karnaugh maps and built circuits relying on these maps. In the end, we were able to design a working prototype in Tinkercad.

The goal was achieved, our model could count from 0 to 15.

Although we could have used less resources like displays, by designing our own decoder, it was too time consuming, thus in order to stick with the deadlines, we had to make some sacrifices along the way, which also includes the maintainability and updatability of our product due to the undocumented and hard to understand design.

References

Dip switch <https://www.endrich.com/sixcms/media.php/2/%20EDS.pdf>
https://www.ti.com/lit/ds/symlink/cd4511b-mil.pdf?ts=1717267011856&ref_url=https%253A%252F%252Fwww.google.com%252F
74HC04 <https://www.diodes.com/assets/Datasheets/74HC04.pdf>
74HC08 <https://www.diodes.com/assets/Datasheets/74HC08.pdf>
74hc32 <https://www.diodes.com/assets/Datasheets/74HC32.pdf>
7 segment display
https://www.idc-online.com/technical_references/pdfs/electronic_engineering/Seven_Segment_Display.pdf
LED [Light-Emitting-Diode.pdf \(csjmu.ac.in\)](#)
Function generator [BK-precision-function-generator-awg-guide.pdf \(mouser.com\)](#)
Resistor
[http://www.ee.ic.ac.uk/pcheung/teaching/DE1_EE/Lectures/Topic%203%20-%20Resistors%20&%20Resistor%20circuits%20\(notes\).pdf](http://www.ee.ic.ac.uk/pcheung/teaching/DE1_EE/Lectures/Topic%203%20-%20Resistors%20&%20Resistor%20circuits%20(notes).pdf)
Diagrams in Figma
<https://www.figma.com/board/MScLABQ3LTIAm8PLTh29be/fsm?node-id=0-1&t=FlpcxQm9oMbC1hwN-1>
Tables in Google Sheet
<https://docs.google.com/spreadsheets/d/1vuacPFJAY5DMVY0qi8wglbmlxDljXc4mzpJcr61dFJw/edit?usp=sharing>
Timing diagram in wavedrom <https://wavedrom.com/tutorial.html>
Logisim for building circuits <https://sourceforge.net/projects/circuit/>